Spécialité Numérique et Sciences Informatiques
EP N°3

Contexte

La mairie d'une ville souhaite optimiser son réseau de bus. Elle dispose d'un fichier CSV contenant les arrêts et les temps de trajet entre eux. Votre mission est d'aider à analyser et améliorer ce réseau.

Structures de données fournies

Pour répondre aux questions, vous disposerez des deux structures de données suivantes, générées à partir du fichier `reseau_bus.csv` :

— **Un dictionnaire des arrêts** : Les clés sont les numéros des arrêts et les valeurs sont leurs noms.

```
arrêts = {
        1: "Cité du Marché 1",
        2: "Boulevard des Fleurs 2",
        5: "Pont de la Montagne 5",
        ...
}
```

— **Un graphe du réseau de bus** : Les clés sont les numéros des arrêts et les valeurs sont des listes de dictionnaires représentant les voisins (avec leur numéro et la distance en minutes).

```
graphe = {1: [{'numero': 5, 'distance': 5},
        {'numero': 96, 'distance': 4}],
        5: [ {'numero': 1, 'distance': 5},
        {'numero': 73, 'distance': 12},
        {'numero': 27, 'distance': 11} ],
        ...
}
```

Travail à réaliser

Première partie : Production autonome

Question 1 : Ajout d'une nouvelle ligne de bus(10 minutes conseillée)

Compétences évaluées : **C** (Comprendre et réutiliser un code existant).

La mairie souhaite ajouter un nouvel arrêt nommé **"Stade Olympique 101"** et le connecter à deux arrêts existants, en considérant que les nouvelles lignes fonctionneront dans les deux sens :

— **Arrêt 1** : "Cité du Marché 1", distance : 8 minutes.

— **Arrêt 5** : "Pont de la Montagne 5", distance : 12 minutes.

Écrire le code nécessaire pour ajouter l'arrêt 101 aux deux structures de données `arrêts` et `graphe`.

Appel 1 **Appeler le professeur pour lui présenter vos réponses ou en cas de difficulté.**

CC BY NC SA

Question 2 : Trouver le nom des arrêts voisins (10 minutes conseillées)

Compétence évaluées : **P** (Concevoir des solutions algorithmiques)

Écrire une fonction `nom_voisins(arrets: dict, graphe : dict, num : int) -> list` qui renvoie une liste des **noms** des arrêts qui sont directement connectés à l'arrêt `num` dont le numéro est passé en paramètre.

Appel 2	**Appeler le professeur pour lui présenter vos réponses ou en cas de difficulté.**

Question 3 : Vérification de l'accessibilité depuis un arrêt (25 minutes conseillées)

Compétence évaluée : **C** (développer des processus de mise au point)

Question 3a : Correction de la fonction de vérification

La fonction suivante est censée vérifier si il existe une possibilité de rejoindre un `arret_arrivee` depuis unn `arret_depart` en utilisant un parcours en profondeur (DFS) itératif. Cependant, elle contient une erreur, elle rentre souvent dans une boucle infinie. Corriger cette fonction.

Fonction à corriger :

```
def existe_chemin(graphe : dict, arret_depart : int, arret_arrivee : int) :
        visites = []
        pile = [arret_depart]

        while pile:
                arret = pile.pop()
                visites.append(arret)
                for voisin in graphe[arret]:
                        if voisin['numero'] == arret_arrivee :
                                return True
                        else :
                                pile.append(voisin['numero'])

        return False
```

assertions :

```
# Exemple de graphe pour les tests
graphe_test = { 1: [{'numero': 2, 'distance': 5}],
        2: [{'numero': 1, 'distance': 5}, {'numero': 3, 'distance': 10}],
        3: [] }

# Série d'asserts sur le graphe test
assert existe_chemin(graphe_test, 2, 1) == True
assert existe_chemin(graphe_test, 2, 3) == True
assert existe_chemin(graphe_test, 3, 1) == False
assert existe_chemin(graphe, 1 , 101) == True
```

Appel 3	**Appeler le professeur pour lui présenter vos réponses ou en cas de difficulté.**

Question 4 : Calcul du plus court chemin avec Dijkstra (15 minutes conseillées)

Pour faciliter la recherche de trajets pour les utilisateurs, la mairie nous met à disposition une fonction `plus_court_chemin(graphe: dict, arret_depart: int, arret_arrivee: int) -> tuple` qui utilise l'algorithme de Dijkstra pour calculer le chemin le plus rapide (sans tenir compte des temps d'attentes aux arrêts) entre deux arrêts passés en paramètres.

La fonction retourne retourner un tuple contenant la durée minimale en minutes du trajet entre deux arrêts et la liste des arrêts formant le chemin.

CC BY NC SA

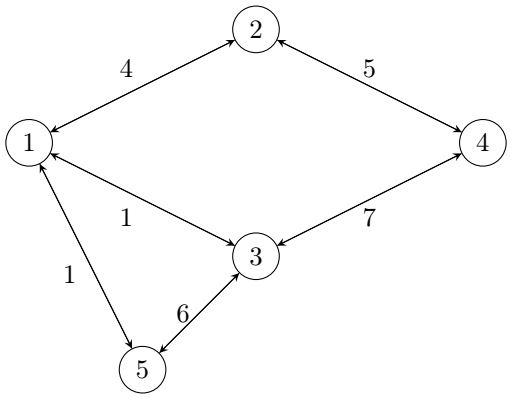
Exemple d'utilisation de la fonction `plus_court_chemin` :

```
>>> plus_court_chemin(graphe, 1, 100)
(23, [1, 96, 77, 3, 56, 68, 100])
```

Question 4a : Assertions

Compétence évaluée : C (développer des processus de validation de programmes.)

Vous devez écrire des assertions pour vérifier que la fonction fonctionne correctement avec le graphe test simplifié fourni ci-dessous




```
# implémentation du graphe pour les tests
graphe_test = {
1: [{'numero': 2, 'distance': 4}, {'numero': 3, 'distance': 1}, {'numero': 5, 'distance': 1}],
2: [{'numero': 1, 'distance': 4}, {'numero': 4, 'distance': 5}],
3: [{'numero': 1, 'distance': 1}, {'numero': 4, 'distance': 7}, {'numero': 5, 'distance': 6}],
4: [{'numero': 2, 'distance': 5}, {'numero': 3, 'distance': 7}],
5: [{'numero': 1, 'distance': 1}, {'numero': 3, 'distance': 6}]
              }
```

Question 4b : l'arrêt le plus loin

Compétence évaluée : P (Concevoir des solutions algorithmiques.)

En utilisant la fonction précédente, écrire une série d'instructions qui permettent de trouver l'arrêt le plus loin de l'arrêt N°1, et le chemin pour y accéder.

Appel 4 **Appeler le professeur pour lui présenter vos réponses ou en cas de difficulté.**